

湖北大学 课程考试试题纸

课程名称: 数据结构 (A 卷)

考试方式: 闭卷 (开卷、闭卷) 印刷份数: _____

学 院: 计算机与信息工程学院 任课教师: 胡书山

专业年级: 信息安全 2021 级

题号	一	二	三	四	五	六	七	八	总分	阅卷教师
得分										

得分 _____ 一、判断分析题(判断下列描述是否正确,并给出分析理由,每小题 3 分,共 15 分)。

1、使用带权有向连通图描述工程预计进度,缩短关键路劲上的某个活动的时间,一定能缩短整个工程的工期。

2、顺序查找算法查找不成功的平均查找长度为 $(n+1)/2$ 。

3、若进栈序列为 1, 2, 3, 4, 5, 6, 且进栈和出栈可以穿插进行,序列 2, 3, 5, 1, 6, 4 是可能出现的出栈序列。

4、数据结构的存储结构包括顺序存储、链式存储、哈希存储三种方式。

5、设某棵二叉树的中序遍历序列为 ABCDE, 先序遍历序列为 CABED, 则后序遍历该二叉树得到序列为 BADCE。

得 分	
--------	--

二、简答题（每小题 5 分，共 25 分）。

1、存在一个有向图 $G=\{V, E\}$ ，顶点数目为 n ，该图的存储结构为邻接表，请回答下列问题：

- (1) 求图 G 中的边数的思路？
- (2) 如何判断图 G 中的两个顶点 i 和 j 是否有边相连？
- (3) 如何计算图中顶点 i 的入度？

2、假设存在一个函数定义：

```
double fun(double x, int n)
{
    if (n==1)
        return x;
    return x* fun(x, n-1);
}
```

- (1) 请问该函数 $\text{fun}(x, n)$ 的功能是？
- (2) 函数调用 $\text{fun}(2, 5)$ 的运算结果是？执行过程中发生了多少次递归调用？
- (3) 函数调用 $\text{fun}(x, n)$ 最多发生了几次递归调用？

3、在以下三种存储结构中，哪个最适合用作链栈？并说明理由。

- (1) 带头节点的单链表

- (2) 不带头节点的循环单链表
- (3) 带头节点的双链表

4、特殊矩阵和稀疏矩阵两种矩阵压缩方法，哪一种方法失去随机存取功能？请解释原因。

5、假设有如下算法，该算法的目标是删除顺序表 L 中的第一个元素，指出其中的错误。

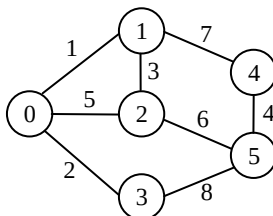
```
void del(Sqlist L)
{
    int i;
    for(i=1; i<L.length; i++)
        L.data[i-1]=L.data[i];
}
```

得分

三、应用分析题(每小题 8 分，共 40 分)。

1、存在一个整数序列{15,5,16,2,25,8,20,9,18,12}，需要对该序列进行升序排序，如果采用二路归并排序法，请写出每一趟排序后的结果。

2、假设有一个带权无向图（如下图所示），以顶点 0 作为起点构造最小生成树，请写出普里姆算法的执行过程。如果使用克鲁斯卡尔算法构造出的最小生成树，请写出执行过程。



3、有一棵二叉排序树 T，该二叉树的先序遍历结果为：(12,5,2,8,6,10,16,15,18,20)。请解答：

- (1) 请构造二叉树 T，并画出树形结构。
- (2) 请写出该二叉树的中序遍历结果。

(3) 求在等概率下的查找成功和不成功情况下的平均查找长度。

4、假设存在一个整数序列{4,5,7,2,1,3,6}，以该序列作为输入，构建一棵空平衡二叉树，请写出该平衡二叉树的构建过程。请画出每个元素插入过程，若需调整，还需给出调整后的结果，并指出调整类型。

5、设有一组关键字{32,13,49,24,38,21,4,12}，其哈希函数为： $H(\text{key})=\text{key} \% 7$ ，采用开放地址法的线性探查法解决冲突，试在 0~9 的哈希地址空间中对该关键字序列构造哈希表，并求等概率下查找成功和查找失败的平均查找长度。

得	
分	

四、程序设计题(每小题 10 分，共 20 分)。

1.假设二叉树采用二叉链存储结构，设计一个算法求二叉树 b 中度为 2 的结点个数，其结点类型如下：

```
typedef struct node
{
    char data;
    struct node *lchild, *rchild;
} BTreeNode;
```

请给出递归模型的设计，并写出函数实现代码。

2、假设存在一个带头结点的单链表 L，每个节点中存储一个整数，请设计一个算法实现：删除 L 中结点值最小的所有结点（可能有多个结点值最小的结点）。

函数声明参考：

DelNode(LinkList * L);